slam_toolbox Mapping Algorithm

1. Algorithm Definition

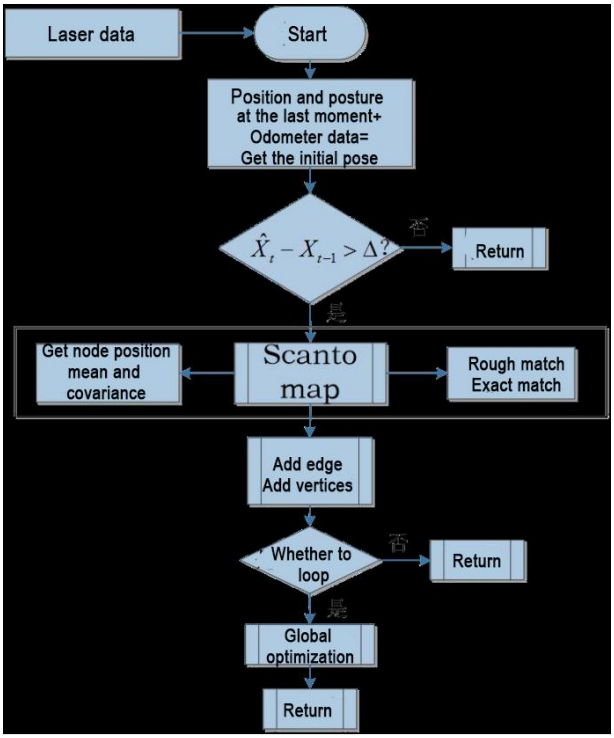
Slam Toolbox software package combines information from laser rangefinders in the form of LaserScan messages and performs TF transformation from odom-> base link to create a two-dimensional map of space. This software package allows for fully serialized reloadable data and pose graphs of SLAM maps, used for continuous mapping, localization, merging, or other operations. It allows Slam Toolbox to operate in synchronous (i.e., processing all valid sensor measurements regardless of delay) and asynchronous (i.e., processing valid sensor measurements whenever possible) modes.

ROS replaces functionalities like gmapping, cartographer, karto, and hector, providing comprehensive SLAM functionality built upon the powerful scan matcher at the core of Karto, widely used and accelerated for this package. It also introduces a new optimization plugin based on Google Ceres. Additionally, it introduces a new localization method called 'elastic pose-graph localization,' which takes measured sliding windows and adds them to the graph for optimization and refinement. This allows for tracking changes in local features of the environment instead of considering them as biases, and removes these redundant nodes when leaving an area without affecting the long-term map.

Slam Toolbox is a suite of tools for 2D Slam, including:

- Mapping, saving map pgm files
- Map refinement, remapping, or continuing mapping on saved maps
- Long-term mapping: loading saved maps to continue mapping while removing irrelevant information from new laser point clouds

- Optimizing positioning mode on existing maps. Localization mode can also be run without mapping using the 'laser odometry' mode
- Synchronous, asynchronous mapping
- Dynamic map merging
- Plugin-based optimization solver, with a new optimization plugin based on Google Ceres
- Interactive RVIZ plugin
- RVIZ graphical manipulation tools for manipulating nodes and connections during mapping
- Map serialization and lossless data storage.



From the above diagram, it can be seen that the process is relatively straightforward. The traditional soft real-time operation mechanism of slam involves processing each frame of data upon entry and then returning.

Relevant source code and WIKI links for KartoSLAM:

- **KartoSLAM ROS Wiki:** http://wiki.ros.org/slam_karto
- **slam_karto software package:**

https://github.com/ros-perception/slam_karto

- **open_karto open-source algorithm:**

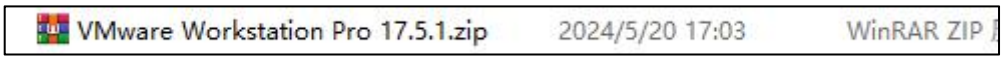
https://github.com/ros-perception/open_karto

2. Mapping Operation Steps

ROS2 mapping and navigation utilize virtual machine connectivity to enable mapping and navigation with the robot on the same local network.

2.1 Install Virtual Machine Software and Import the Virtual Machine

1) Unzip the provided installation files, then click on the installation file to proceed with the installation.



2) Click on  to open the virtual machine.

3) Enter the virtual machine interface and click on "Open Virtual Machine".

4) Select the image file provided in the tutorial and import it. For specific instructions, please refer to the lesson “11.Depth Camera Basic Lesson/Test and Configure ROS2”.

5) After the import is completed, follow the prompts to complete the installation process.

2.2 Copy Robot Files to the Virtual Machine

◆ **Export files from the robot**

Note: If the folders ros2_ws and src.zip already exist in the Home directory of the virtual machine, please delete them first before proceeding with the following steps in the tutorial.

---

1) Start the robot, and access the robot system desktop using VNC.

2) Click-on  to open the ROS2 command-line terminal.

3) Execute the command to disable the app auto-start service.

```
~/.stop_ros.sh
```

4) Enter the command to navigate to the ros2_ws/src/ directory:

```
cd ros2_ws/src/
```

5) Enter the command to package the three files 'navigation', 'slam', and 'simulations' into a compressed file:

```
zip -r src.zip navigation slam simulations
```

6) Enter the command to move the compressed file to the shared directory:

```
mv src.zip /home/ubuntu/shared
```

7) Run the command and return to ros2_ws directory:

```
cd ..
```

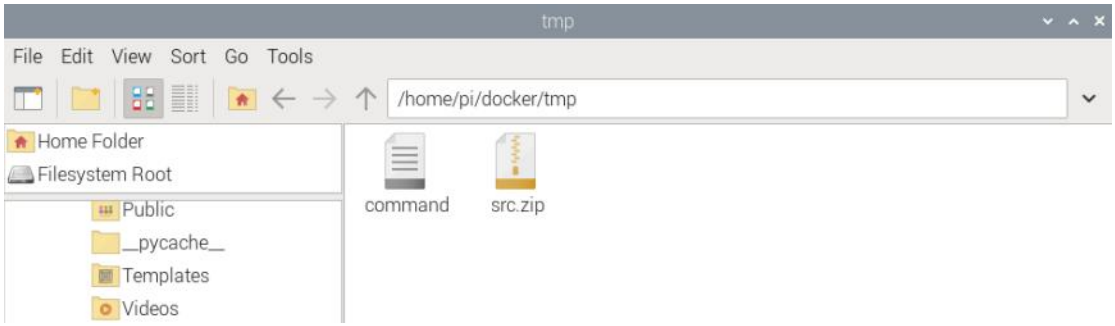
8) Enter the command to view the .typerc file in this directory:

```
ls -a
```

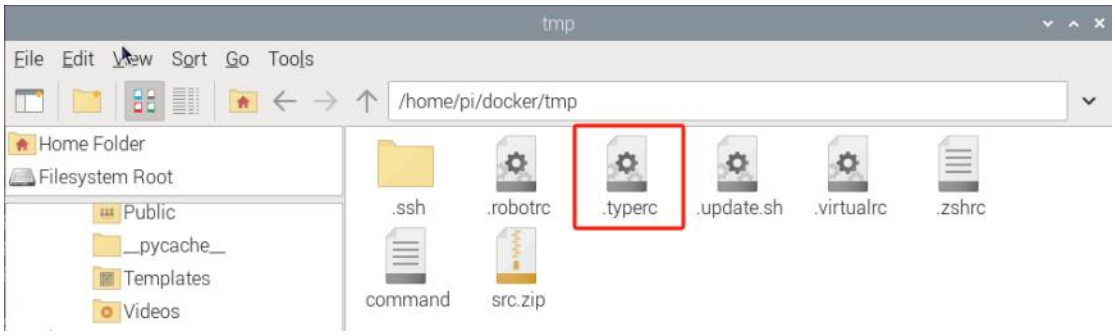
9) Enter the command to move the .typerc file to the shared directory:

```
cp .typerc /home/ubuntu/shared
```

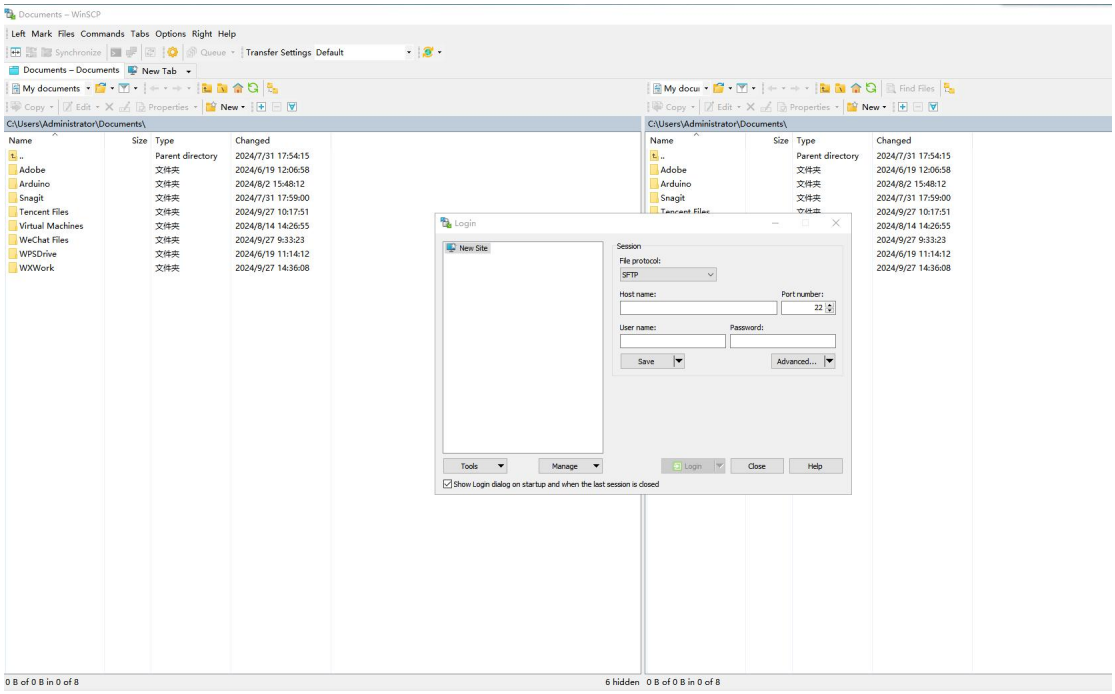
10) Click-on  to open the file directory, then navigate to the home/pi/docker/tmp directory:



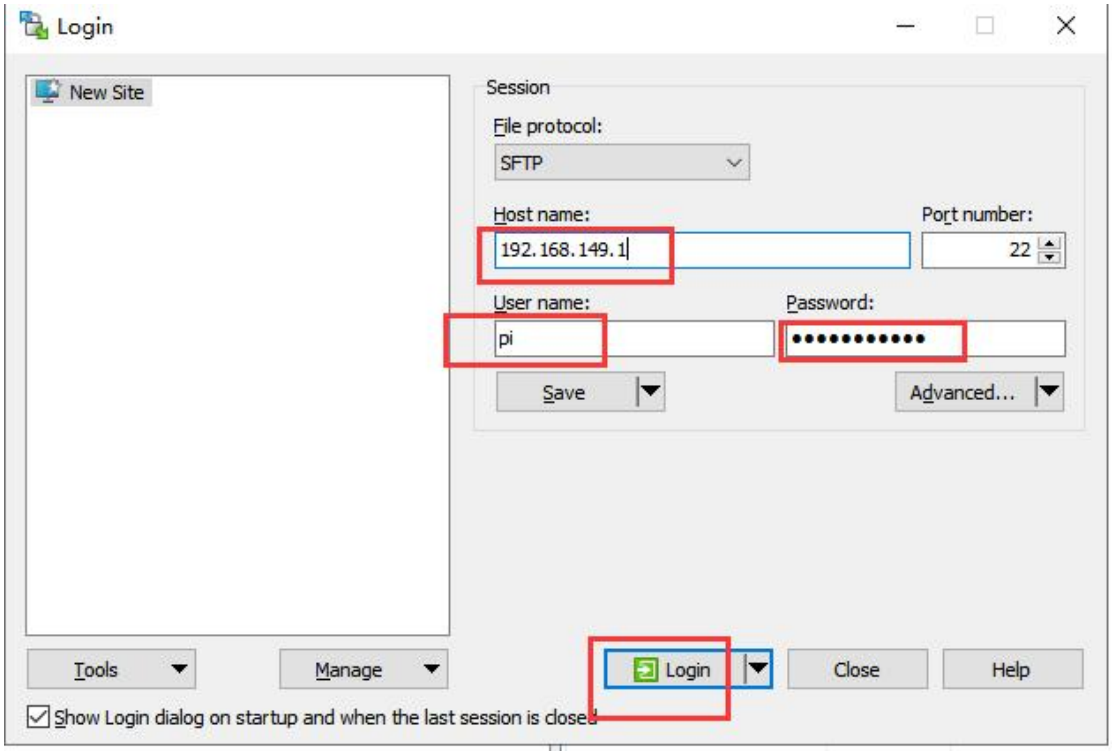
11) Use the shortcut 'Ctrl + H' to show the hidden .typerc file:



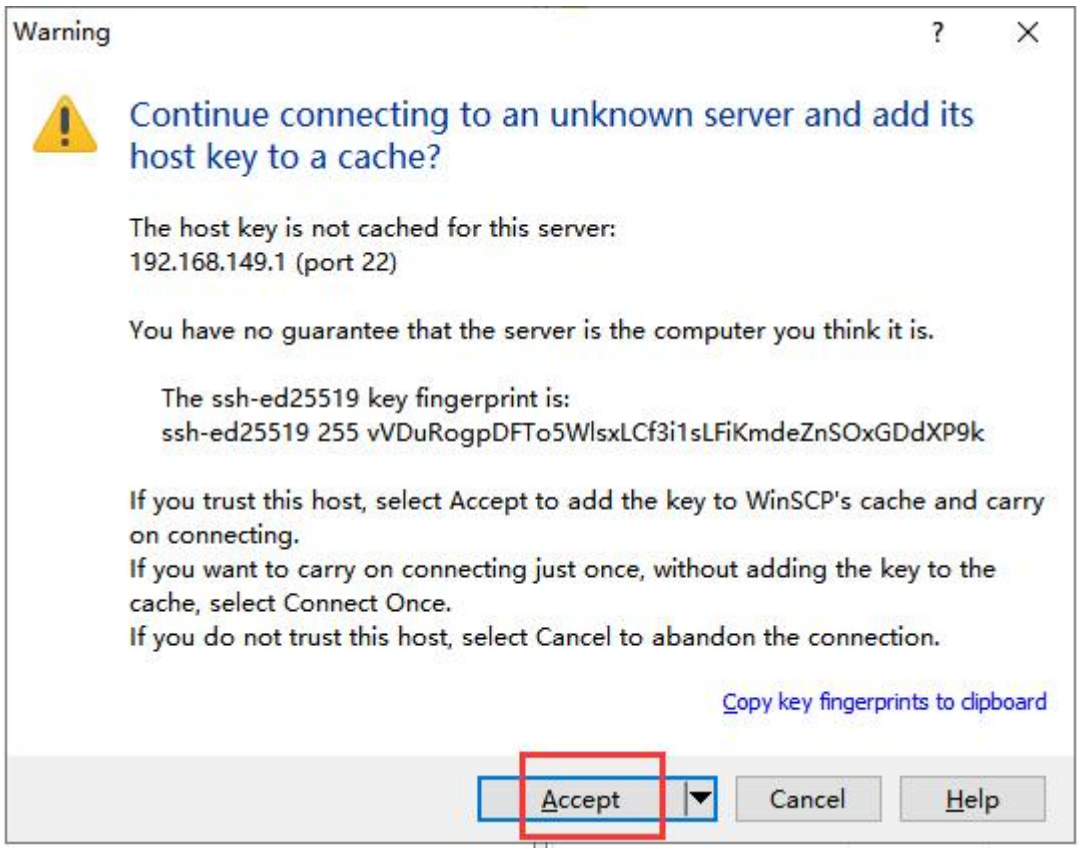
12) Refer to the instruction in “Install WinSCP” to install and open the WinSCP tool.



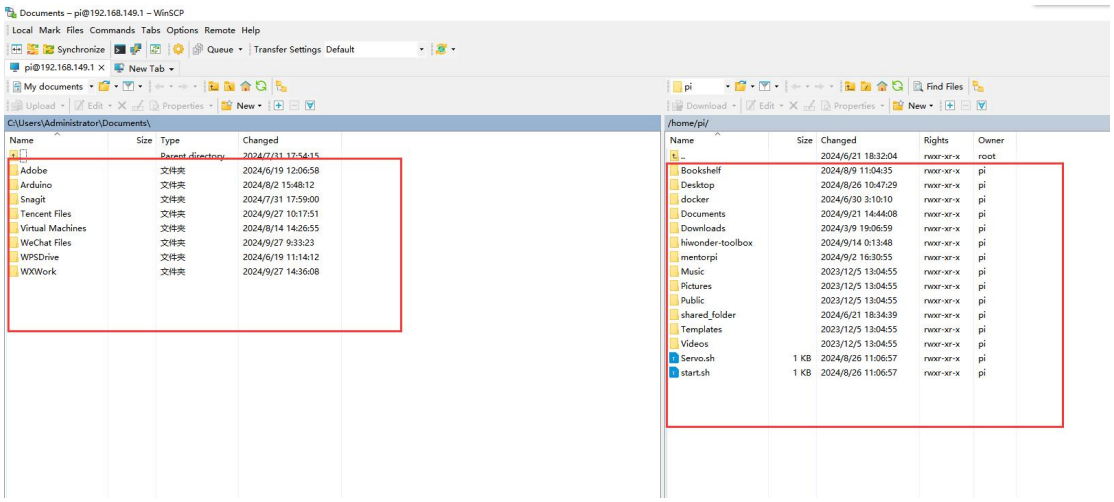
13) Enter the IP address of Raspberry Pi “192.168.149.1”, username “pi”, and password “raspberrypi”. Then click “Login”.



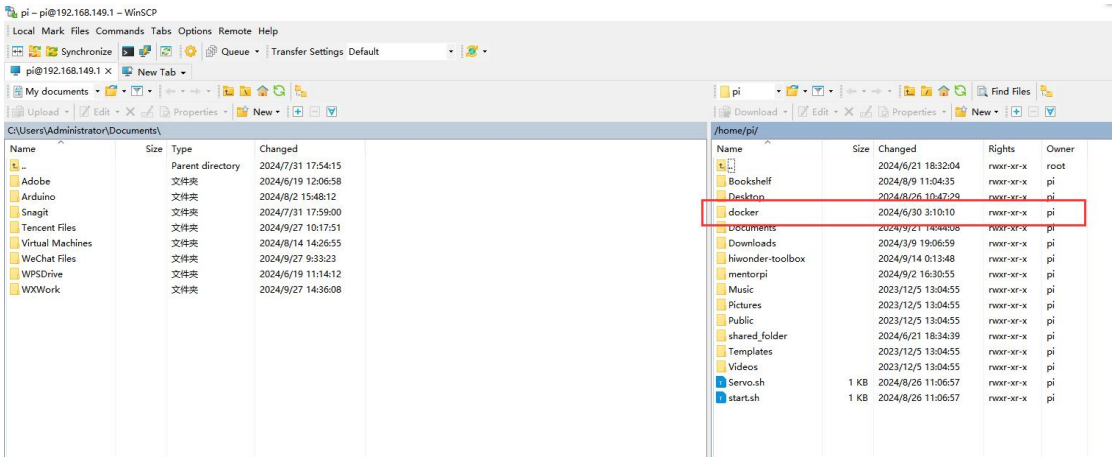
14) Click “Accept” on the pop-up window that appears.



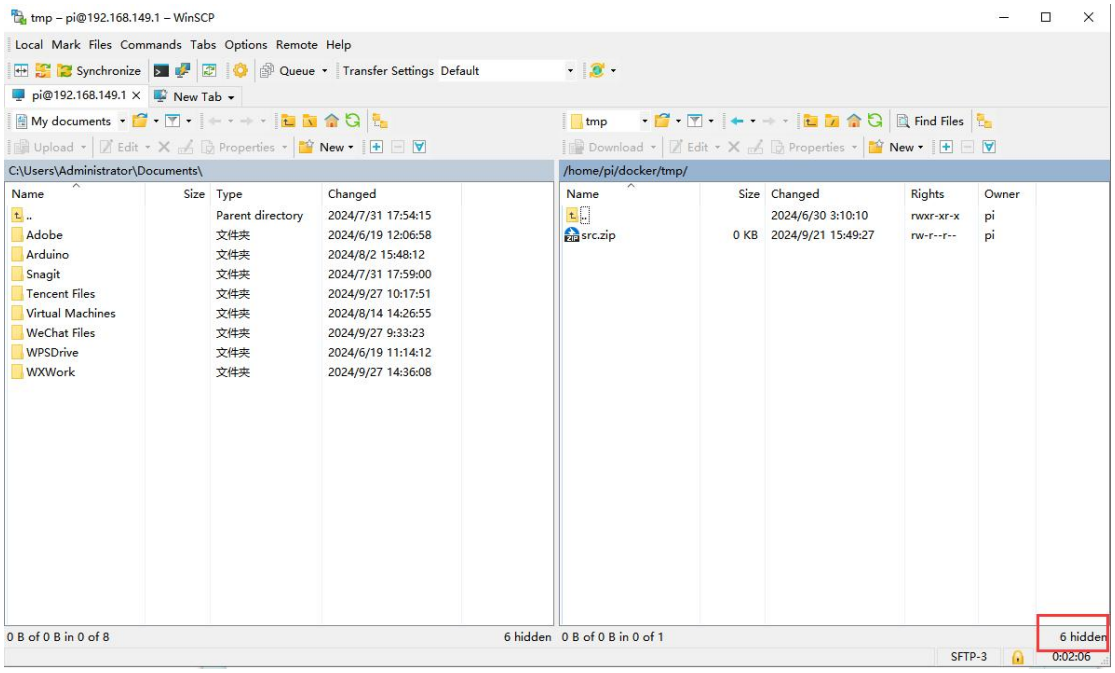
15) The left side shows the directory of your computer, and the right side shows the root directory of Raspberry Pi.



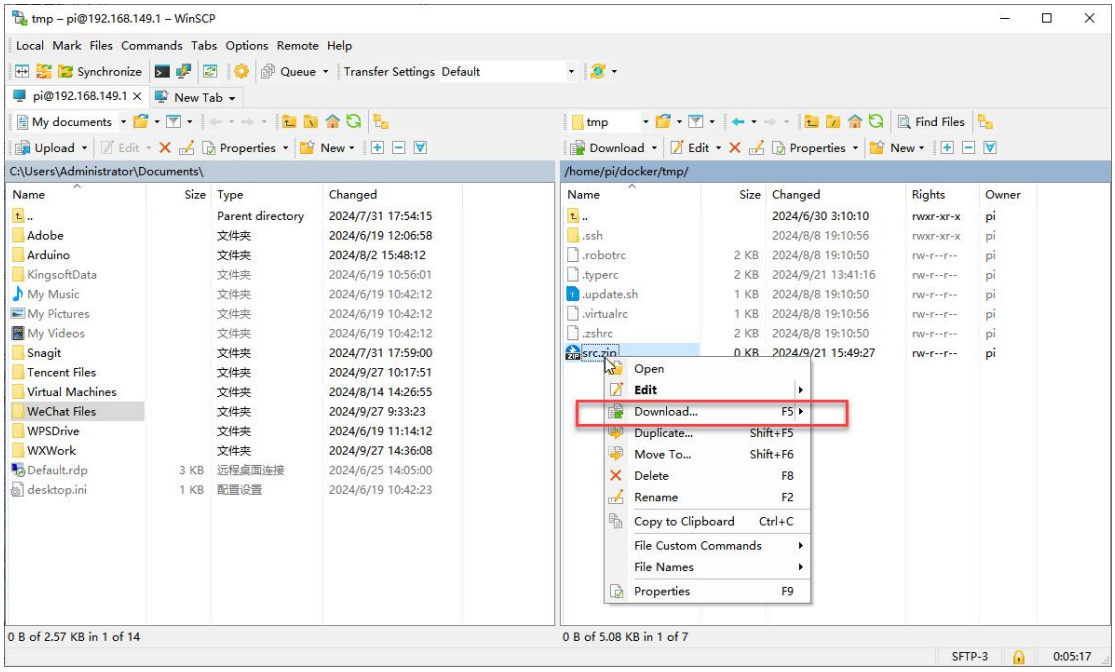
16) Select the folder "docker->tmp" and open the shared folder with Docker.



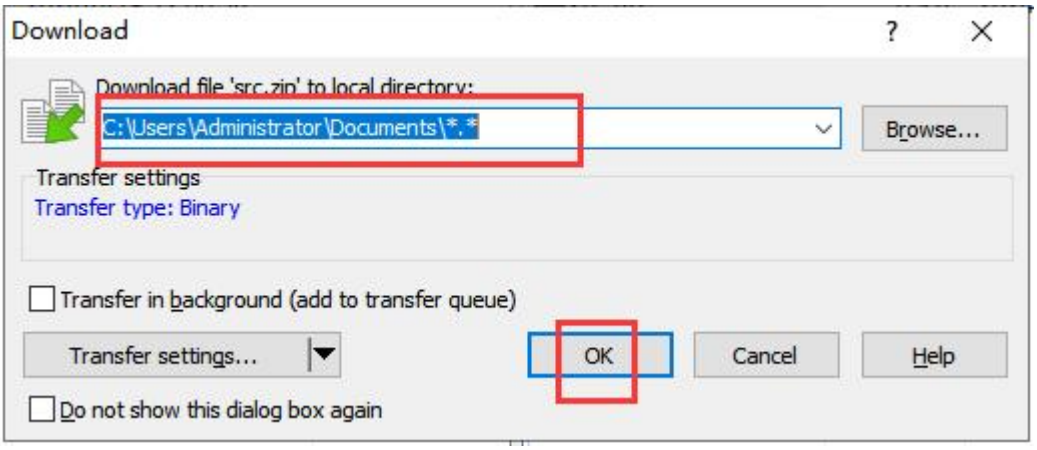
17) In the bottom right corner of WinSCP, there is a number indicating the number of hidden files. Double-click on it to display hidden files.



18) Select the "src" folder and right-click to choose "Download".



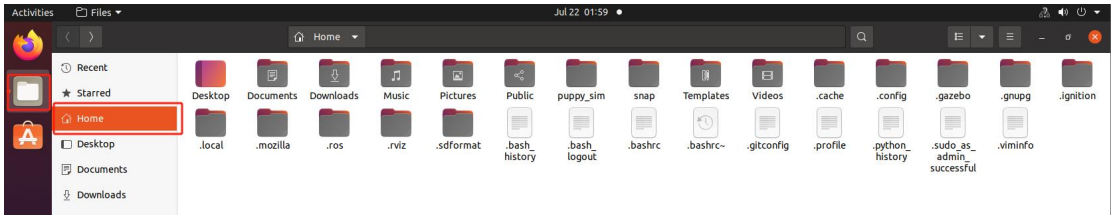
19) Select the path where you want to save the file and click "OK".



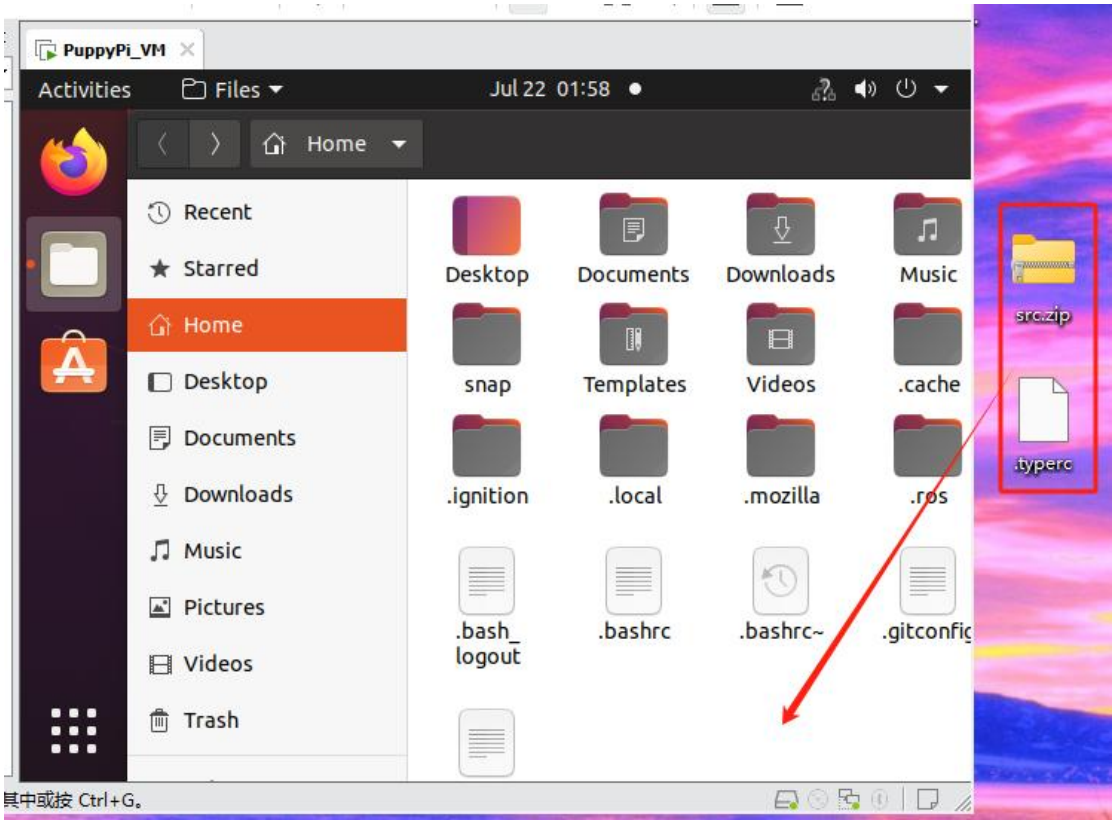
20) Follow step 18) to save the .typerc file.

◆ Import files into the virtual machine

1) Click-on  to navigate to the home directory.



2) Drag and drop the .typerc and src.zip files from the computer desktop into the virtual machine:



2.3 Create and Compile the Workspace

1) Click-on  to start the virtual machine command-line terminal.

2) Enter the command to create the ros2_ws/src directory:

```
mkdir -p ros2_ws/src
```

```
ubuntu@ubuntu-virtual-machine:~$ mkdir -p ros2_ws/src
```

3) Extract the file “src.zip” to the directory “home/ubuntu”.

unzip src.zip

```
ubuntu@ubuntu-virtual-machine:~$ unzip src.zip
```

4) Move the simulations, slam, and navigation files to the ros2_ws/src directory:

mv simulations slam navigation /home/ubuntu/ros2_ws/src/

```
ubuntu@ubuntu-virtual-machine:~$ mv simulations slam navigation /home/ubuntu/ros2_ws/src/
```

5) Move the .typerc file to the ros2_ws directory:

mv .typerc ros2_ws/

```
ubuntu@ubuntu-virtual-machine:~$ mv .typerc ros2_ws/
```

6) Navigate to the ros2_ws directory:

cd ros2_ws/

```
ubuntu@ubuntu-virtual-machine:~$ cd ros2_ws/
```

7) Enter the command to check if .typerc has been moved to the ros2_ws directory:

ls -a

```
ubuntu@ubuntu-virtual-machine:~/ros2_ws$ ls -a
.  ..  src  .typerc
```

8) Input the following command to compile the workspace:

colcon build

```
ubuntu@ubuntu-virtual-machine:~/ros2_ws$ colcon build
```

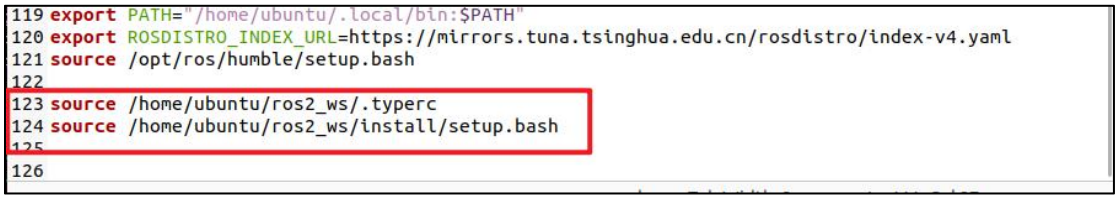
9) Change the .bashrc file, and input the following command:

gedit ~/.bashrc

```
ubuntu@ubuntu-virtual-machine:~/ros2_ws/src/slam/launch$ gedit ~/.bashrc
```

10) Copy the content below to the file .bashrc.

```
source /home/ubuntu/ros2_ws/.typerc
source /home/ubuntu/ros2_ws/install/setup.bash
```



11) After finishing writing, use the shortcut **Ctrl + S** or click the **Save** button in the upper right corner to save and exit.

12) Run the command below to refresh the environment configuration.

```
source ~/.bashrc
```

2.4 Set the Robot to LAN Mode

Notice

1) When MentorPi starts up in LAN mode, it will automatically search for the preset network. During this process, LED2 on the Raspberry Pi expansion board will flash rapidly, indicating that it is searching. If the target network is not found after three attempts, MentorPi will automatically switch to Direct Connection mode (LED2 flashes slowly). In this mode, you can connect directly to the hotspot provided by MentorPi.

---

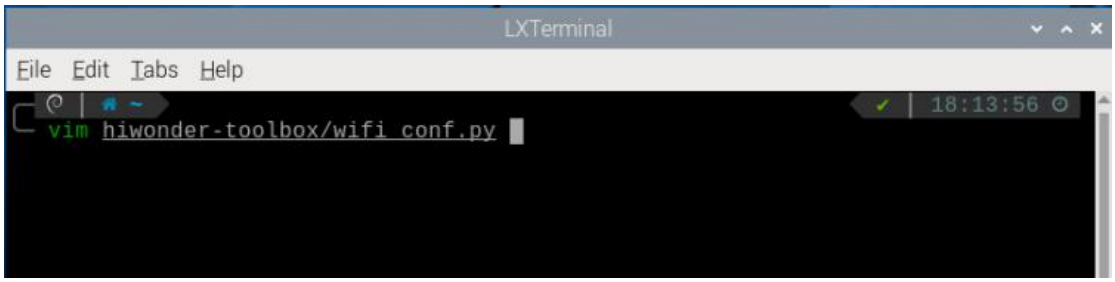
2) If MentorPi has been set to connect in LAN mode to a specific device's hotspot, but that device is temporarily unavailable, you may use another device. Simply configure the new device's hotspot with the same name and password as the original one, and MentorPi will be able to connect automatically.

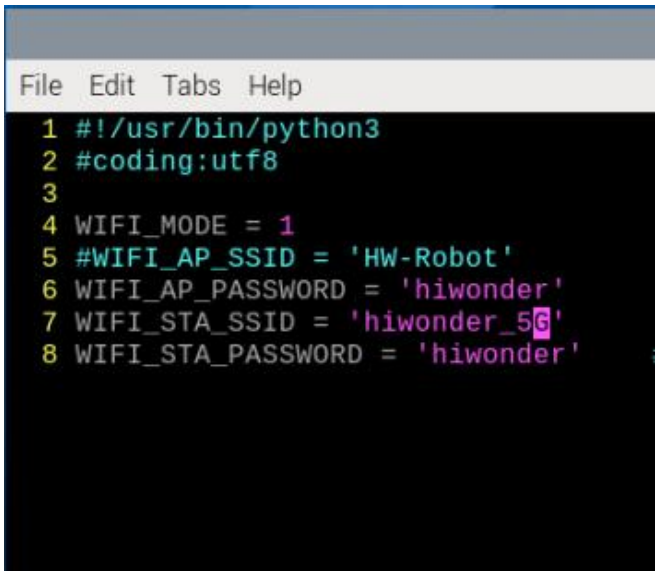
3) When MentorPi is in LAN mode, it will not create its own hotspot.

1) Click-on  to open the command line terminal.

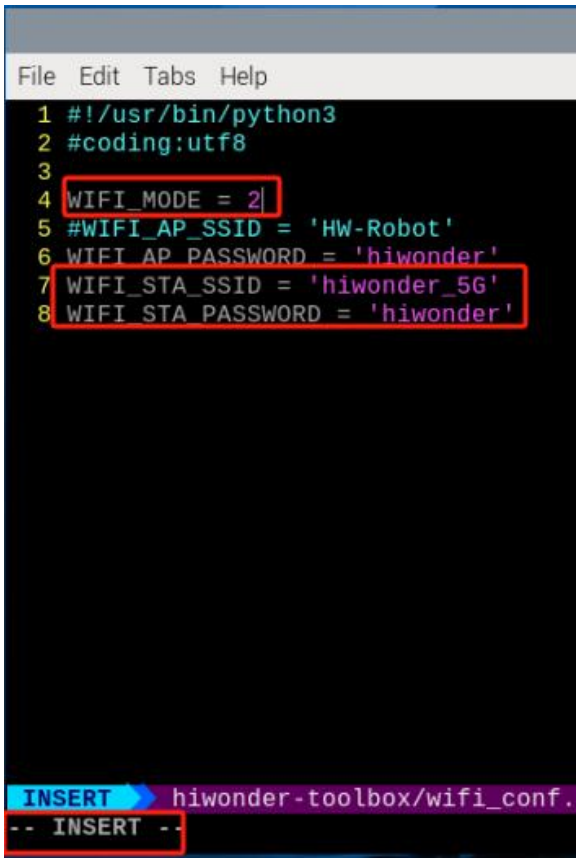
2) Enter the command to open the WiFi configuration file.

```
vim hiwonder-toolbox/wifi_conf.py
```





3) Press "i" key to enter the editing mode. Change the WiFi configuration file to LAN mode, and modify your own WiFi name and password.



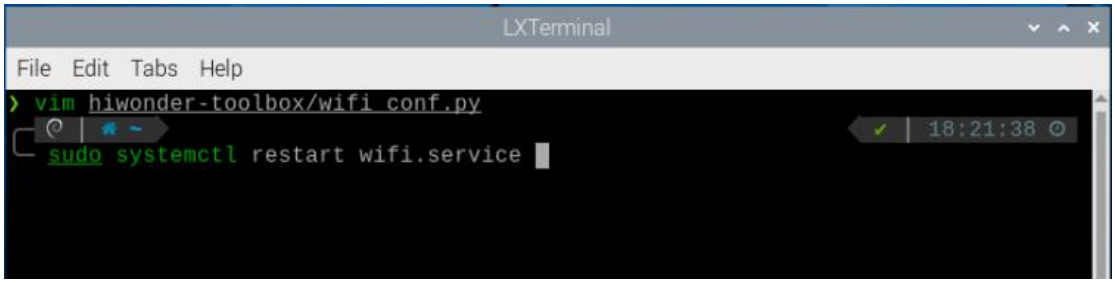
Note: When MentorPi fails to find the target network in LAN mode and automatically switches back to Direct Connection mode, the content of wifi_conf.py will not be modified automatically. It will still retain the LAN configuration. If no changes are made, MentorPi will continue to start in LAN mode by default the next time it is powered on.

4) After finishing writing, press “Esc” and enter the command to save and exit.

:wq

5) Enter the command to restart the network, or reboot the system. It is recommended to reboot the system.

sudo systemctl restart wifi.service



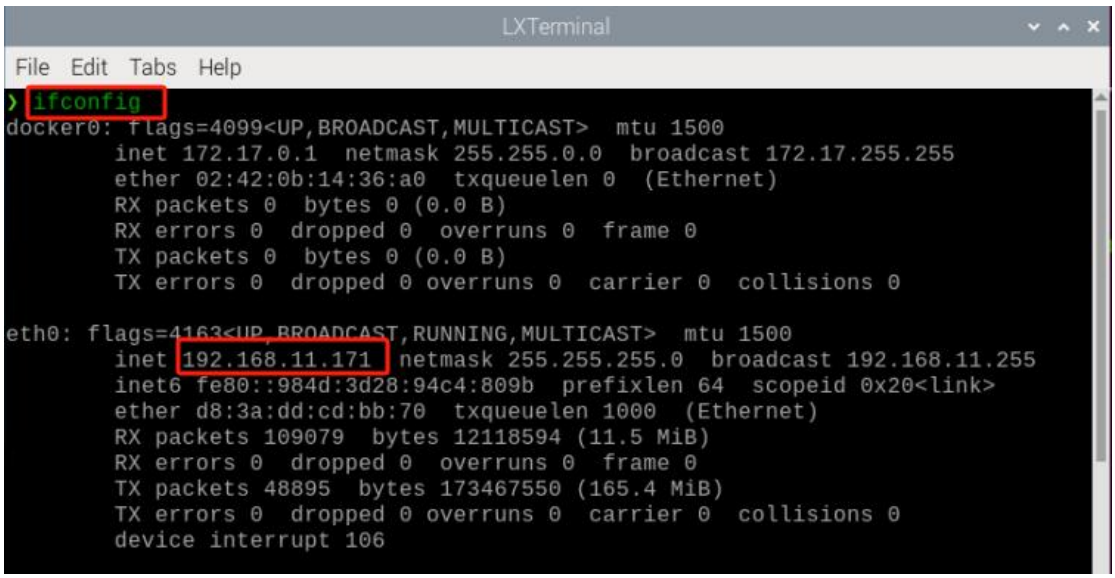
6) Refer to “1. Getting Ready -> Lesson 4 APP Control” to obtain the robot's LAN IP address through the app connection.

7) It is important to note that the virtual machine and the robot are connected to the same LAN, and their IP addresses should be within the same subnet:

Virtual Machine:

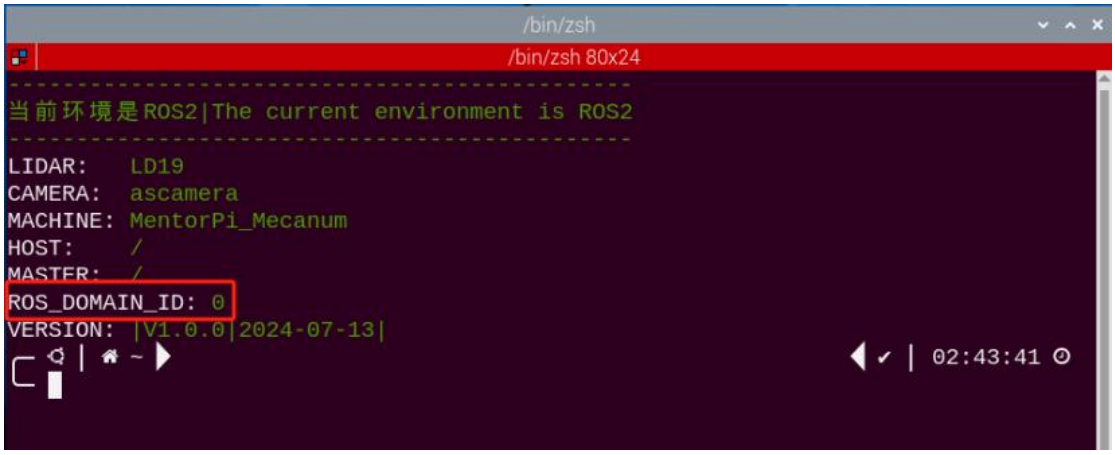
```
ubuntu@ubuntu-virtual-machine:~/ros2_ws$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.11.109  netmask 255.255.255.0  broadcast 192.168.11.255
        inet6 fe80::44ac:39cd:5685:bd4b  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:8d:f9:35  txqueuelen 1000  (Ethernet)
        RX packets 216730  bytes 189191130 (189.1 MB)
        RX errors 0  dropped 36  overruns 0  frame 0
        TX packets 45822  bytes 4134681 (4.1 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 184  bytes 19693 (19.6 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 184  bytes 19693 (19.6 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Robot:



8) After rebooting, click on  to open the robot's ROS2 command line terminal. You will notice that the ROS_DOMAIN_ID is 0, which is the same as on the virtual machine.

Robot:



Virtual Machine:

3. slam Mapping Instructions

◆ **Robot operations**

1) Click-on  to open the command-line terminal.

2) Execute the following command to disable the app auto-start service.

```
~/.stop_ros.sh
```

3) Run the command to initiate mapping:

```
ros2 launch slam slam.launch.py
```

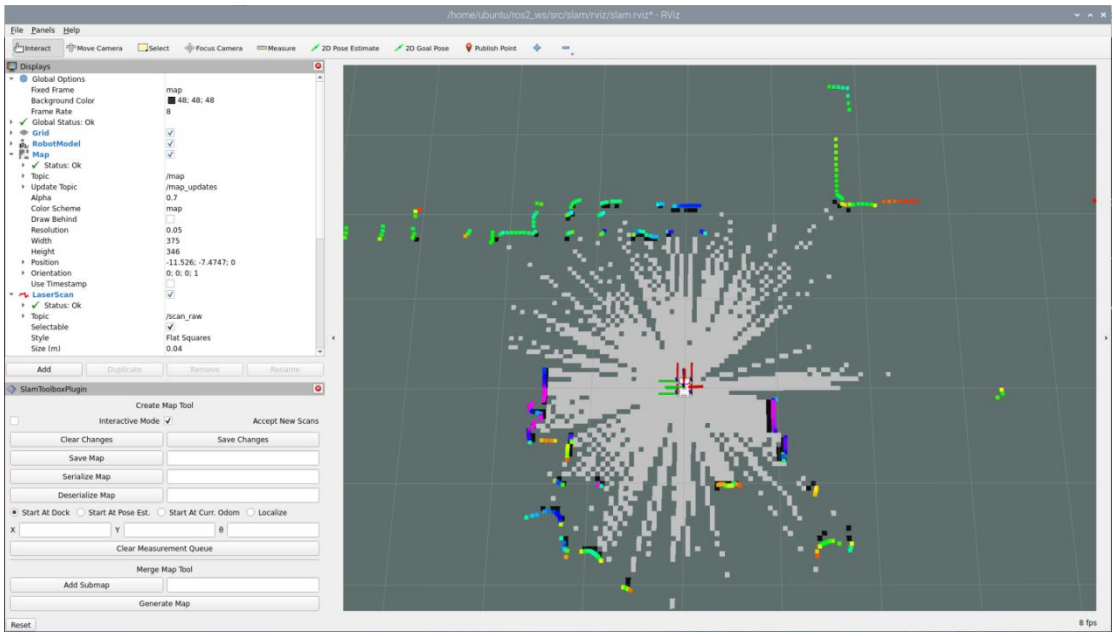
◆ **Virtual Machine instructions**

1) Click-on  to open the command line terminal of the virtual machine system.

2) Enter the command to open the RViz tool and display the mapping results.

ros2 launch slam rviz_slam.launch.py

```
ubuntu@ubuntu-virtual-machine:~$ ros2 launch slam rviz_slam.launch.py
```



◆ **Enable keyboard control**

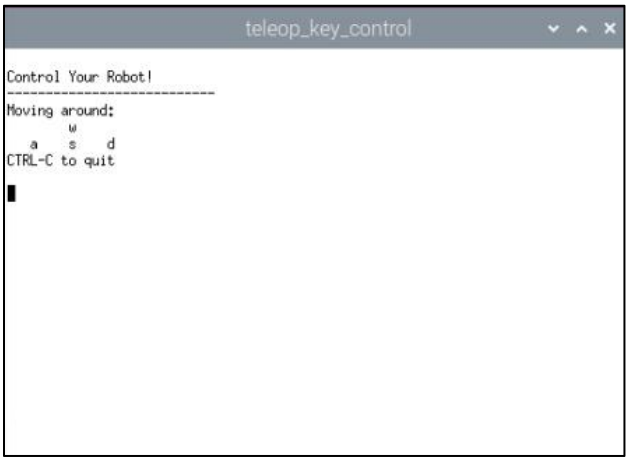
1) Click-on  to open the command-line terminal.

2) Enter the command to start the keyboard control node, and press “Enter”.

ros2 launch peripherals teleop_key_control.launch.py

```
~ | 02:43:41
ros2 launch peripherals teleop_key_control.launch.py
```

If you see the prompt as shown in the following image, it means the keyboard control service has been successfully started.



3) Control the robot to move in the current space to build a more complete map. The table below lists the keyboard keys available for controlling robot movement and their corresponding functions:

Key	Robot Action
W	Short press to switch to forward state and continuously move forward
S	Short press to switch to backward state and continuously move backward
A	Long press to interrupt the forward or backward state and turn left
D	Long press to interrupt the forward or backward state and turn right

4) When controlling the robot's movement with the keyboard to map, it's advisable to reduce the robot's movement speed appropriately. The slower the robot's speed, the smaller the odometry relative error, leading to better mapping results. As the robot moves, the map displayed in RVIZ will continuously expand until the entire environment scene's map construction

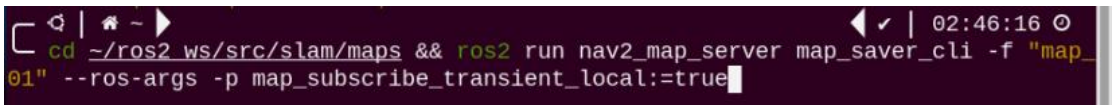
is completed.

4. Save Map

1) Click-on  to open the command-line terminal.

2) Run the following command to save the map.

```
cd ~/ros2_ws/src/slam/maps && ros2 run nav2_map_server
map_saver_cli -f "map_01" --ros-args -p
map_subscribe_transient_local:=true
```



3) If you want to exit the game, press “Ctrl+C” in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

```
sudo systemctl restart start_node.service
```

5. Effect Optimization

If you desire a more precise mapping outcome, optimizing the odometry can be beneficial. Mapping with the robot requires the use of odometry, which in turn relies on the IMU.

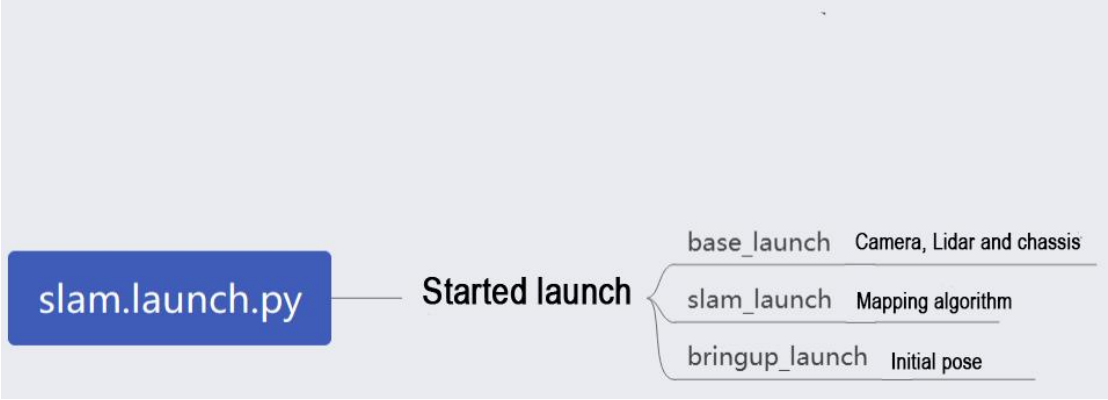
The robot itself comes with pre-calibrated IMU data loaded, enabling it to perform mapping and navigation functions effectively. However, calibrating the IMU can still enhance accuracy further. The calibration method and steps for the IMU can be found in the "9. Motion Control Lesson -> Lesson 3 IMU, Linear Velocity and Angular Velocity Calibration" section.

6. Parameter Explanation

The parameter file can be found in the "ros2_ws\src\slam\config\slam.yaml" directory.

For more detailed information about the parameters, please refer to the official documentation: https://wiki.ros.org/slam_toolbox

7. Launch File Analysis



The launch file is located at:

/home/ubuntu/ros2_ws/src/slam/launch/slam.launch.py

◆ Import Library

The launch library can be explored in detail in the official ROS documentation:

https://docs.ros.org/en/humble/How-To-Guides/Launching-composable-nodes.html

```python
import os
from ament_index_python.packages import get_package_share_directory

from launch_ros.actions import PushRosNamespace
from launch import LaunchDescription, LaunchService
from launch.substitutions import LaunchConfiguration
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, GroupAction, OpaqueFunction, TimerAction
```

◆ Set the Storage Path

Use the “get_package_share_directory” function to obtain the path of the slam package.

```python
    if compiled == 'True':
        slam_package_path = get_package_share_directory('slam')
    else:
        slam_package_path = '/home/ubuntu/ros2_ws/src/slam'
```

◆ Initiate Other Launch File

```python
    base_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(slam_package_path, 'launch/include/robot.launch.py')),
        launch_arguments={
            'sim': sim,
            'master_name': master_name,
            'robot_name': robot_name
        }.items(),
    )

    slam_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(slam_package_path, 'launch/include/slam_base.launch.py')),
        launch_arguments={
            'use_sim_time': use_sim_time,
            'map_frame': map_frame,
            'odom_frame': odom_frame,
            'base_frame': base_frame,
            'scan_topic': f'{frame_prefix}scan_raw',  # Using scan_raw topic
            'enable_save': enable_save
        }.items(),
    )

    if slam_method == 'slam_toolbox':
        bringup_launch = GroupAction(
            actions=[
                PushRosNamespace(robot_name),
                base_launch,
                TimerAction(
                    period=6.0,
                    actions=[slam_launch],
                ),
            ]
        )
```

base_launch: Launch for hardware initialization

slam_launch: Launch for basic mapping

bringup_launch: Launch for initial pose setup